TP 06 — Programmation Shell

CO	Description	Ce TP
CO1	Understand OS fundamentals, Linux command-line environment, and operating system service interfaces	—
CO2	Understand process scheduling, memory management, and file system organization algorithms	—
CO3	Manage Linux users and permissions, implement shell scripts, and apply text processing tools	✓

Contexte

Vous êtes administrateur système junior dans une PME qui gère un petit parc de serveurs (web, base de données, cache, messagerie). Votre responsable vous demande d'**automatiser** les tâches répétitives en écrivant des scripts shell : un script de bienvenue qui valide ses arguments, un script d'**inventaire** qui interroge la liste des machines. Un collègue vous laisse au passage un script « presque fini » mais truffé de bugs à débusquer. La mission se déroule en trois objectifs. À la fin de chaque objectif, un **script interactif** valide automatiquement votre travail et génère un rapport que vous montrez à l'enseignant.

Tableau de mission

Objectif	Mission	Script	Récompense
0	Préparer le poste de travail	— (PDF)	5 pts
1	Écrire son premier script (structure, variables, paramètres)	`lab06_ex1.sh`	5 pts
2	Automatiser l'inventaire : conditions, boucles, fonctions, options	`lab06_ex2.sh`	5 pts
3	Déboguer un script	`lab06_ex3.sh`	5 pts

Objectifs pédagogiques

À la fin de ce TP, vous serez capable de :

* Écrire un script shell (shebang, variables, paramètres, statut de sortie)
* Valider les arguments et gérer les codes de retour
* Utiliser conditions, boucles, fonctions et options
* Manipuler les noms de fichiers avec l'expansion de paramètres
* Déboguer un script avec la trace d'exécution et la vérification de syntaxe

Pré-requis et matériel

* Terminal Bash fonctionnel (WSL2 sur Windows, Terminal.app sur macOS, ou émulateur natif Linux)
* L'éditeur `micro` installé (voir Exercice 0 ; `micro -help` pour les raccourcis)
* Avoir assisté au cours CM06 (scripts, variables, conditions, boucles, fonctions, débogage)
* Avoir réalisé les TP1 à TP5
* L'archive `scripts_lab06.zip` téléchargée depuis le site du cours (voir Exercice 0)
* Les fichiers fournis dans l'archive : `machines.txt`, le squelette `inventory.sh`, le script bogué `script_debug.sh`

Déroulement et barème

Le TP se joue en **3 objectifs** de 25 à 30 minutes chacun. Pour chaque objectif, vous lancez le script correspondant (`./lab06_ex1.sh`, `./lab06_ex2.sh`, `./lab06_ex3.sh`) : il vous guide étape par étape, vérifie vos réponses (ou l'état réel de votre machine), vous donne des indices en cas d'erreur, puis génère un **rapport** (`report_lab06_exN.txt`) que vous montrez à l'enseignant.

Pour chaque objectif, ouvrez **deux terminaux** dans le répertoire du TP : l'un pour lancer le script, l'autre pour effectuer le travail (écrire vos scripts avec `micro`).

À la fin de **chaque objectif**, vous rédigerez sur papier une **synthèse libre manuscrite** (« ce que j'ai fait, pourquoi ça marche, le lien avec le cours ») — c'est elle qui est notée, à raison de 5 points par objectif.

Barème (sur 20) : Exercice 0 « Environnement prêt » (5 points) + trois objectifs notés sur 5 points chacun (15 points). Absence de machine opérationnelle = 0/20, quel que soit le reste.

Annexe — Aide-mémoire des commandes du TP

Ce tableau ne remplace pas le cours. En cas de doute, utilisez `man commande` ou `commande --help`.

Écrire et exécuter un script

Commande	Syntaxe	Rôle
Shebang	`#!/bin/bash`	Première ligne : indique l'interpréteur
`micro`	`micro fichier.sh`	Éditeur (`Ctrl+S` sauvegarder, `Ctrl+Q` quitter)
`chmod`	`chmod +x script.sh`	Rendre le script exécutable
Exécution	`./script.sh`	Lancer en tant que programme (nécessite `chmod +x`)
`$1` `$2`	`echo "$1"`	Paramètres positionnels (arguments)
`$#`	`[ $# -eq 0 ]`	Nombre d'arguments
`$?`	`echo $?`	Statut de sortie de la dernière commande
`$()`	`d=$(date)`	Substitution de commande (capture la sortie)
`exit`	`exit 0` / `exit 1`	Quitter avec un code (0 = succès)
`bash -x`	`bash -x script.sh`	Trace d'exécution (chaque commande après expansion)
`bash -n`	`bash -n script.sh`	Vérifier la syntaxe sans exécuter
`man -k`	`man -k mot-clé`	Rechercher une commande par mot-clé

`man -k` s'appuie sur la base `whatis` : si elle ne renvoie rien, lancez `sudo mandb` (Linux) ou `sudo makewhatis` (macOS).

Tests, conditions et boucles

Commande	Syntaxe	Rôle
`[ -f f ]`	`[ -f fichier ]`	Vrai si `fichier` existe et est un fichier
`[ -d d ]`	`[ -d dossier ]`	Vrai si `dossier` est un répertoire
`[ -x f ]`	`[ -x fichier ]`	Vrai si `fichier` est exécutable
`-eq` `-ne`	`[ a -eq b ]`	Comparaisons numériques (`-ne`, `-lt`, `-gt`)
`if`	`if condition; then ...; fi`	Branchement conditionnel
`case`	`case "$1" in a) ... ;; esac`	Aiguillage multiple (fermé par `esac`)
`for`	`for f in *.txt; do ...; done`	Boucle sur une liste
`while read`	`while read -r ligne; do ...; done < f`	Lire un fichier ligne par ligne
`$((...))`	`n=$((n+1))`	Calcul arithmétique (pas d'espace autour de `=`)

Fonctions, options et manipulation de chaînes

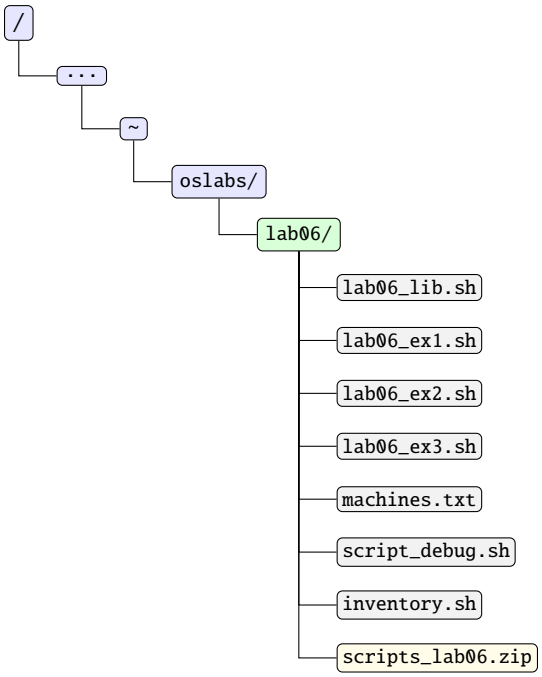
Commande	Syntaxe	Rôle
Fonction	`nom() { ... }`	Définir une fonction (appelée par `nom`)
`local`	`local x="$1"`	Variable limitée à la fonction
`getopts`	`while getopts "v" opt; do ...; done`	Analyser les options de la ligne de commande
`shift`	`shift $((OPTIND-1))`	Décaler les paramètres après `getopts`
`$var%`	`$f%.*`	Retirer le suffixe le plus court (l'extension)
`$var%%`	`$1%%.*`	Retirer le plus long suffixe (le domaine)
`$var##`	`$f##*.`	Retirer le plus long préfixe (garde l'extension)
`--help`	`commande --help`	Aide rapide

1 Exercice 0 — Préparer le poste de travail

Durée : 15 min — 5 points « Environnement prêt »

Contexte

Avant toute mission, on prépare son poste. Ces commandes reviennent à **chaque TP** : à force de répétition, elles deviendront des réflexes. Pour ce TP, où vous allez **écrire vos propres scripts**, on installe en plus l'éditeur `micro` et on crée le dossier `outils` qui recevra votre travail. À la fin de cet exercice, votre arborescence doit ressembler à ceci :



Instructions

1. Aller à la racine : `cd /`
2. Revenir au répertoire personnel : `cd ~` ou `cd` seul
3. Entrer dans le dossier commun : `cd oslabs`
4. Créer le dossier du TP : `mkdir lab06`
5. Entrer dans le dossier du TP : `cd lab06`
6. Installer l'éditeur : `sudo apt update && sudo apt install micro`
   (repli Ubuntu : `sudo snap install micro` — macOS : `brew install micro`)
7. Vérifier l'éditeur : `micro -version`
8. Télécharger l'archive :
   `curl -O https://liaoux.github.io/it-intro-to-os/static_files/labs/lab06/scripts_lab06.zip`
9. Extraire les fichiers : `unzip scripts_lab06.zip`
10. Rendre les scripts exécutables : `chmod +x *.sh`

11. Créer le dossier de travail : `mkdir outils`
12. Vérifier le contenu du dossier : `ls`

Attendu : les fichiers du schéma — `lab06_lib.sh`, `lab06_ex1.sh`, `lab06_ex2.sh`, `lab06_ex3.sh`, `machines.txt`, `inventory.sh`, `script_debug.sh` et `scripts_lab06.zip` — ainsi que le dossier `outils/`.

`curl -O` est disponible par défaut sur Linux, macOS et WSL ; il télécharge l'archive dans le dossier courant. `micro` n'a pas de page `man` : consultez `micro -help` (et `micro -help keys` pour les raccourcis — `Ctrl+S` sauvegarder, `Ctrl+Q` quitter).

Appelez le professeur pour valider votre environnement avant de commencer les objectifs.

2 Objectif 1 — Écrire son premier script

Durée : 25 min — 5 points

Vous écrivez votre premier script de zéro : sa structure (shebang, commentaires), l'exécution (en tant que programme, via l'interpréteur ou dans le shell courant), les variables, la substitution de commande, le statut de sortie, les paramètres positionnels et la validation des arguments.

Lancez le script (terminal 1) : `./lab06_ex1.sh`

3 Objectif 2 — Automatiser l'inventaire du parc

Durée : 30 min — 5 points

Vous complétez un squelette fourni pour automatiser l'inventaire du parc : une vérification de fichier, un menu, deux fonctions, une boucle de lecture sur la liste des machines et une option de la ligne de commande.

Lancez le script (terminal 1) : `./lab06_ex2.sh`

4 Objectif 3 — Débogage

Durée : 30 min — 5 points

Vous débusquez cinq bugs glissés dans un script fourni à l'aide de la trace d'exécution et de la vérification de syntaxe, avant une relecture par les pairs.

Lancez le script (terminal 1) : `./lab06_ex3.sh`

Appelez le professeur avec vos rapports `report_lab06_exN.txt` en main, accompagnés d'une synthèse libre manuscrite (notes personnelles) : ce que vous avez fait, pourquoi ça marche, et le lien avec le cours.